

neuro-san-esp

An agent network that searches for a better agent network, and keeps doing it without being asked.

neuro-san can turn a sentence into a working multi-agent network. It produces **one** network and never measures it — there is no fitness function anywhere in the framework. Nobody knows whether a nine-agent topology beats a five-agent one for the same job, which model each agent should run, or whether the generated instructions are any good. That is design without evaluation.

This repository adds the missing half: a way to score a network, a search for a better one, and a service that runs that search unattended. The method is Cognizant AI Lab's own — **Evolutionary Surrogate-assisted Prescription**: learn a cheap Predictor from real measurements, evolve thousands of candidates against it for free, and spend expensive real evaluations only on the elite.

The one number that explains the design

Evaluating one candidate network for real takes about eight minutes of language-model time. Scoring one with the Predictor takes **0.064 milliseconds**. Two thousand candidates cost 0.13 seconds against the surrogate; the same two thousand evaluated for real would take roughly **267 hours**. That ratio is the whole reason this is ESP and not a plain genetic algorithm — and it is measured on this repository, not quoted from a paper.

What is actually here

A genome	neuro-san's own <code>agent_network_definition</code> , plus a per-agent model. Reusing the framework's representation is what makes mutation free.
A fitness function	Accuracy, tokens, latency and agent count, measured on 17 multi-hop questions over a synthetic 124-document corpus.
Seven mutation operators	<code>add</code> , <code>remove</code> , <code>rewire</code> , <code>split</code> , <code>merge</code> , <code>toggle_search</code> , <code>reassign_model</code> — each gated by a validity check, invalid mutants discarded not repaired.
A Predictor	Gradient-boosted regression over structural and textual genome features, retrained after every batch of real evaluations.
A service	An event-invoked neuro-san agent on an hourly schedule that spends the day's provider budget, writes down what it learned, and stays silent unless it found something better.

The gap this fills

Why the evolutionary machinery was already there, unconnected.

Reading neuro-san-studio closely, almost every component an evolutionary loop needs is already implemented for other reasons. What is missing is the one piece that turns them into a search.

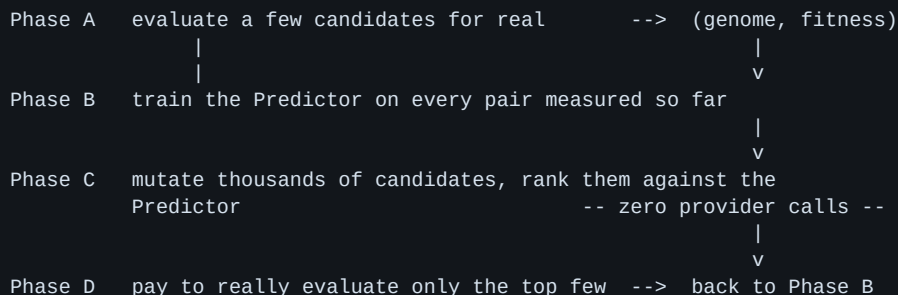
Evolutionary component	Already in neuro-san
Genome	agent_network_definition — round-trips to HOCON through HoconAgentNetworkAssembler
Mutation operators	coded_tools/agent_network_editor/ — add_agent, remove_agent, update_agent
Viability check	StructureNetworkValidator — cycles, unreachable nodes, DAG shape
Deploy a candidate without restart	reservations → BranchActivation.use_reservation()
Candidate garbage collection	ExpiringAgentNetworkStorage — TTL and LRU
Baseline to beat	the designer's own one-shot output
A fitness function	nothing. Anywhere.

Stated honestly: the idea of searching agent architectures is not new

ADAS uses a meta-agent over an archive, AFlow runs Monte-Carlo tree search over operator graphs, GPTSwarm learns edge probabilities with reinforcement learning, and MaAS and Promptbreeder attack neighbouring problems. None of them targets neuro-san, and none uses surrogate-assisted evolution. The claim made here is the method applied to this framework and measured on it — not the invention of architecture search.

How it works

Four phases. Phase C is the point.



In plain language

Phase A. Run three starting networks against 17 real questions and record how accurate each was, how many tokens it burned, and how many agents it used. This costs real money and real time: about eight minutes a network.

Phase B. Fit a small model that reads a network's shape — agent count, depth, branching, which model each agent runs, how long its instructions are — and predicts how well it will score. It is trained on tens of samples, so it is weak. Its job is not to be right; its job is to *rank*.

Phase C. Generate thousands of mutated networks and score every one with the Predictor. No language model is called, so this is effectively free. Two thousand candidates take a tenth of a second.

Phase D. Take only the best few the Predictor picked and evaluate those for real. Feed the results back into Phase B, which now has more data, and repeat.

A real-world analogy

Testing a car design in a wind tunnel is slow and expensive, so engineers fit a cheap simulator to the tunnel results, try ten thousand shapes in the simulator overnight, and put only the best handful back in the tunnel. The tunnel is Phase D. The simulator is the Predictor. This repository is that loop, with agent networks instead of car bodies.

The mutation operators

Operator	What it changes	Why it matters
add_agent / remove_agent	how many agents exist	removal re-parents the orphans instead of severing them
rewire	one edge, size unchanged	isolates topology from size
split_agent / merge_agents	granularity	the axis a designer told to 'use the fewest agents' can never explore
toggle_search	which agents may read the corpus	retrieval placement is a real design decision
reassign_model	per-agent model tier	the main cost/quality knob, and nobody tunes it today

Every mutant is checked — one top agent, a directed acyclic graph, no unreachable agents, no dangling references, at least one agent able to search — and **discarded if it fails, never repaired**. A repaired mutant

is a different mutant from the one the operator produced, and silently substituting one would leave the search unable to learn what its own operators do.

Every file, and what it is for

The repository, annotated: 112 tracked files, plus 20 package markers and cache fixtures listed only as this count.

(root)

File	What it does
<code>.dockerignore</code>	<i>supporting file</i>
<code>.env.example</code>	<i>supporting file</i>
<code>.gitignore</code>	<i>supporting file</i>
<code>Dockerfile</code>	The service image.
<code>LICENSE</code>	<i>supporting file</i>
<code>Makefile</code>	Every command in this document has a target here.
<code>README.md</code>	<i>supporting file</i>
<code>SECURITY.md</code>	Keys are read from the environment and never committed.
<code>SERVING.md</code>	How to deploy the service, and what is verified about it.
<code>compose.yaml</code>	The service, with its state volume.
<code>pyproject.toml</code>	<i>supporting file</i>

.devcontainer

File	What it does
<code>devcontainer.json</code>	<i>supporting file</i>

.github

File	What it does
<code>ci.yml</code>	Lint, tests and a real offline search on every push.

apps

File	What it does
<code>README.md</code>	<i>supporting file</i>
<code>run_optimizer.py</code>	Run one wake from the command line or a container, no server needed.
<code>serve.py</code>	<i>supporting file</i>

deploy

File	What it does
Dockerfile	<i>supporting file</i>
README.md	<i>supporting file</i>

docs

File	What it does
FINDINGS.md	<i>supporting file</i>
neuro-san-esp-Dossier.pdf	<i>supporting file</i>
neuro-san-esp-Explainer.pdf	<i>supporting file</i>
neuro-san-esp-Primer.pdf	<i>supporting file</i>
neuro-san-esp-Verification.pdf	<i>supporting file</i>
index.json	<i>supporting file</i>
lint.txt	<i>supporting file</i>
log.txt	<i>supporting file</i>
offline_search.txt	<i>supporting file</i>
periodic_server.txt	<i>supporting file</i>
serving_champion.txt	<i>supporting file</i>
studio.txt	<i>supporting file</i>
tests.txt	<i>supporting file</i>
tree.txt	<i>supporting file</i>
01-web-client.png	<i>supporting file</i>
02-question-typed.png	<i>supporting file</i>
03-agent-session.png	<i>supporting file</i>
fresh-clone-chrome.png	<i>supporting file</i>
web-01-loaded.png	<i>supporting file</i>
web-02-answered.png	<i>supporting file</i>

esp

File	What it does
config.py	<i>supporting file</i>
corpus_tool.py	The retrieval tool the candidates share. Deterministic term overlap, three documents.

File	What it does
failover.py	Moves to the next model when one model's daily quota is spent.
measurements.py	<i>supporting file</i>
smoke.hocon	<i>supporting file</i>
ratelimit.py	Paces every provider call underneath neuro-san, because a 429 would otherwise be scored as a bad topology.
runner.py	Runs a candidate in-process, measures accuracy, tokens and time, and caches by genome hash.
tasks.py	17 questions of one to four hops, with exact answers and the scorer.
world.py	A synthetic logistics company: 24 depots, 40 contracts, 60 incidents, 124 documents, one seed.
loop.py	Population, selection, elitism, the multi-objective fitness function and the Pareto front.
definition.py	The genome. A neuro-san agent network as an object that can be hashed, compared and rendered back to HOCON.
mutations.py	The seven operators that change a network, and the validity gate every mutant must pass.
seeds.py	The three starting topologies, including a faithful reconstruction of what neuro-san's own designer produces.
build.py	The run report.
dossier.py	This document.
explainer.py	<i>supporting file</i>
facts.py	<i>supporting file</i>
layout.py	Shared page furniture for both PDFs.
plots.py	Fitness curve, surrogate scatter, Pareto front.
primer.py	The same project with no jargon, for a reader who has never seen it.
coded_tools.py	The neuro-san CodedTool that lets an agent run a wake.
optimizer.py	One wake: spend what today allows, write it down, stop cleanly.
preflight.py	<i>supporting file</i>
state.py	The population and today's budget, written to disk after every single evaluation so an interruption costs nothing.
predictor.py	The Predictor. Genome features to predicted fitness -- the part that makes this ESP rather than a genetic algorithm.

registries

File	What it does
manifest.hocon	The hourly schedule. This is the line that makes it a service.
optimizer.hocon	The autonomous agent itself -- event-invoked, with instructions to stay quiet unless something improved.

results

File	What it does
fitness.png	<i>supporting file</i>
pareto.png	<i>supporting file</i>
surrogate.png	<i>supporting file</i>
history.json	<i>supporting file</i>

scripts

File	What it does
adopt_measurements.py	<i>supporting file</i>
baseline_report.py	The seed measurements, as text.
capture_proofs.py	Runs the commands whose output appears in this document.
offline_search.py	Phases B and C only. Zero provider calls, so it runs with no key at all.
probe_models.py	Which models are usable today and what budget is left.
run_esp.py	The batch search, for when budget is available in bulk.
serve_champion.py	Writes the best-measured topology into the registry so a person can talk to it in a browser.
serve_studio.py	<i>supporting file</i>
service_report.py	Turns the service's accumulated population into the report inputs.
smoke_inprocess.py	Proves a network can be run in-process with no server.
sync_history.py	<i>supporting file</i>
verification_report.py	<i>supporting file</i>
verify_periodic.py	Starts a real neuro-san server and proves it fires the optimiser with no user and no client attached.

state

File	What it does
champion.json	<i>supporting file</i>

tests

File	What it does
test_coded_tools.py	<i>supporting file</i>
test_committed_measurements.py	<i>supporting file</i>

File	What it does
test_container.py	supporting file
test_devcontainer.py	supporting file
test_docs.py	supporting file
test_eval_and_surrogate.py	supporting file
test_failover.py	supporting file
test_genome.py	supporting file
test_ground_truth.py	supporting file
test_lease_and_clock.py	supporting file
test_live_quota_payloads.py	supporting file
test_model_config.py	supporting file
test_mutations.py	supporting file
test_offline_search.py	supporting file
test_packaging.py	supporting file
test_preflight.py	supporting file
test_proofs.py	supporting file
test_provider_switch.py	supporting file
test_ratelimit.py	supporting file
test_ratelimit_keyring.py	supporting file
test_ratelimit_max_retries.py	supporting file
test_report.py	supporting file
test_report_facts.py	supporting file
test_service.py	supporting file
test_service_report.py	supporting file
test_service_surrogate.py	supporting file
test_studio.py	supporting file
test_task_outcomes.py	supporting file
test_web.py	supporting file

From a script to a service

The part that makes this production rather than a demo.

The first version of this project was a batch script: plan four generations, run them, print a report. It died every time — not from a bug, but because the provider's free tier allows 500 requests per day per model and one candidate costs about 165 of them. A day's budget buys three candidates. A four-generation run needs forty.

The reframe the whole design turns on

The daily cap is not an obstacle to a service. It is a service's **rhythm**. Spend what today allows, write down where you got to, stop, and carry on tomorrow. A population accumulates over weeks that could never be bought in an afternoon — and every one of those weeks is unattended.

What one wake does

- | | |
|------------------------------|--|
| 1. take the lease | one optimiser at a time, or two wakes |
| 2. read yesterday's state | spend the same budget twice |
| 3. prime the failover ladder | population, and which models are spent today |
| 4. Phase C, free | so this fresh process does not rediscover |
| 5. evaluate up to 3 | this morning's exhaustion by paying for it |
| 6. quota hit? | rank 400 mutants with the Predictor |
| 7. release the lease | saving state after EVERY candidate |
| | that is a normal ending, not an error |
| | in a finally block; and it expires anyway |

Every design decision here exists because of a specific failure

Decision	The failure it prevents
State written after every candidate, not every generation	An interruption after an eight-minute evaluation would otherwise throw that evaluation away — and interruption is the expected ending.
Atomic write (temp file, then rename)	A service killed mid-write comes back to a truncated population file and has lost everything.
A lease with a timeout	Two overlapping wakes spend the same daily budget twice and write over each other. A lease with no timeout wedges the service forever the first time a wake is killed.
A corrupt lease is treated as free	Better to risk one overlapped wake than to need a human to delete a file before the service runs again.
Exhaustion recorded per <i>day</i> , not permanently	A model retired for good would leave the service dead after its first bad afternoon — the opposite of running unattended.
Quota failures are never cached as scores	A daily cap fails tasks part-way through a candidate, producing a plausible partial score. Cached, it tells the search forever that a good network is mediocre.
The agent stays silent unless something improved	Most wakes find nothing. A service that announces every wake trains its operator to ignore it, and then the one wake that matters is ignored too.
The agent has no tool that deletes or sends	An autonomous process running hourly with no human in the loop should not be able to do anything irreversible or outward-facing.

The line that makes it a service

```
"optimizer.hocon": {  
  "serve": true,  
  "periodic": {  
    "interactions": [{  
      "enable": true,  
      "cron_schedule": "0 * * * *",  
      "metadata": {"user_id": "system"},  
    }],  
  },  
}
```

Hourly, not every fifteen minutes. A wake evaluates up to three candidates and the free tier buys about three a day, so a faster schedule would only produce wakes that find the budget already spent and decline. The cadence is set by what the provider allows, not by how often we would like news.

Proof

Captured by running the commands, not written by hand.

Every transcript on this page was produced by `scripts/capture_proofs.py`, which runs the real command and writes its real output to `docs/proofs/`. This document reads them from there. If a command starts failing, the proof changes and so does this page.

The test suite

```
$ /home/user/neuro-san-esp/.venv/bin/python -m pytest tests -q — exit 0
```

```
..... [ 27%]  
..... [ 54%]  
..... [ 81%]  
..... [100%]  
264 passed in 14.91s
```

Unit tests across the genome, the mutation operators, the evaluator, the surrogate, the failover ladder and the service. Several of them are regressions for bugs listed later in this document.

Lint

```
$ /home/user/neuro-san-esp/.venv/bin/python -m ruff check esp tests scripts apps — exit 0
```

```
All checks passed!
```

Ruff over every package, with the rule set pinned in `pyproject.toml` rather than left at the tool's defaults — default drift turned CI red once already.

Proof, continued

The part that costs nothing and proves the most.

Phases B and C, with no provider key at all

```
$ /home/user/neuro-san-esp/.venv/bin/python scripts/offline_search.py --pool 2000 — exit 0
```

```
No local measurements in .esp-cache -- falling back to the committed ones in tests/fixtures/cache.
Run `make baseline` with a key to train on your own instead.
```

```
Phase B -- training the Predictor on 3 real evaluations from tests/fixtures/cache
```

```
surrogate on 3 samples: rank quality NOT MEASURED -- cross-validation needs 8. The predictor is untrained and returns
```

```
Phase C -- evolving 2000 candidates against it
```

```
2000 viable, 822 rejected as invalid
```

```
scored in 0.12s with zero provider calls
```

```
~0.058 ms per candidate
```

```
!! The surrogate is UNTRAINED on 3 samples (needs 8). Every prediction below is the same constant, so this is NOT a
```

```
First 5 in generation order -- all tied, see above:
```

```
+0.7842 915e149aed588d57 agents=5 depth=2 via split_agent
```

```
+0.7842 0334802d6e9b1376 agents=2 depth=2 via merge_agents
```

```
+0.7842 7edd0c338d311ac0 agents=5 depth=3 via add_agent
```

```
+0.7842 9f1d2fa76d71da2e agents=4 depth=2 via split_agent
```

```
+0.7842 0334802d6e9b1376 agents=2 depth=2 via merge_agents
```

```
Phase C scored 2000 candidates in 0.1s. The same number evaluated for real, at roughly 8 minutes each, would take about
```

```
Phase C generated and scored them for nothing, which is the cost claim. It did not rank them, and it never measures -
```

This is the ESP claim, executed. Two thousand candidate networks generated, validated, and ranked by the Predictor with zero calls to any language model. The rejected count is the validity gate doing its job.

Read the surrogate line honestly

At the generation the search actually used it — nine real samples — the Predictor's cross-validated rank correlation was **−0.333: worse than chance**, and recorded in results/history.json rather than hidden. Refitted afterwards on all twelve it comes out positive on every split tried, roughly **+0.24 to +0.65** — but that is a measurement taken after the fact, and at twelve samples the figure moves with the cross-validation split, so no single value from it is worth quoting. The machinery is correct and the ranking is free; whether the ranking is any good is a function of how many real evaluations have accumulated, which is exactly why the service runs every hour instead of once.

The framework really does wake it

```
$ /home/user/neuro-san-esp/.venv/bin/python scripts/verify_periodic.py — exit 0
```

```
2026-08-24T03:32:01 user_id=None Starting PeriodicEventInitiator with 1.000000 seconds period
2026-08-24T03:32:01 user_id=None Found 1 periodic agent interactions
2026-08-24T03:32:01 user_id=None HealthProbeServer started on port 8081
2026-08-24T03:33:00 user_id=system Received a optimizer.StreamingChat request
2026-08-24T03:34:00 user_id=system Received a optimizer.StreamingChat request
2026-08-24T03:35:00 user_id=system Received a optimizer.StreamingChat request
```

```
--- 3 scheduled fires, one per minute, no client connected ---
```

A real neuro-san server, started with this repository's manifest and the cron shortened to once a minute. **user_id=system with no client attached** is the property that matters: the framework initiated those interactions by itself. Produced by scripts/verify_periodic.py, which fails if the fires do not arrive.

What this proof does not cover

A wake completing a full evaluation *inside* the server process. The front agent needs its own provider call before it can reach the tool, and on an exhausted free tier that call returns 429 first. The trigger is what is verified here; the wake itself is verified separately and repeatedly by `apps/optimizer/run_optimizer.py`, which runs identical code. Reporting this as a verified end-to-end run would be exactly the overstatement the rest of this repository is built to avoid.

Proof, continued

Commit history

`$ git log --oneline -15` — `exit 0`

```
bbc5ac9 Rotate keys before retiring a model on per-day 429
74a3c10 feat: the service can now serve the winner it found
```

Fifteen most recent commits.

Talking to the champion

The measured best topology, served and opened in a browser.

A search that ends with a winning hash in a report has stopped one step short. The reason to measure topologies is that one of them is better to actually *use*, so `scripts/serve_champion.py` writes the current best genome into the registry as an ordinary neuro-san agent network. It is regenerated and gitignored rather than hand-maintained: a champion edited by hand would drift from the genome that earned the score, and the thing being served would stop being the thing that was measured.

(assembled by hand -- two captures: `serve_champion.py` re-run on a clean clone 2026-09-11, and one live 2026-08-22 session with a real neuro-san server, curl and a browser) — captured out of band, not by the harness

```
Two captures, from different dates, because the two halves need different
things: the first runs anywhere, the second needed a live server and provider
budget. Both are shown as taken.

--- captured 2026-09-11, on a clean clone with no key and no service state

$ python scripts/serve_champion.py
No service population yet -- serving the best committed measurement: mut:reassign_model at +0.8453.
Run a wake with a key to serve your own.

champion: mut:reassign_model (6859dda0dfabcf2d)
  accuracy 0.88   tokens 260,052   agents 5   fitness +0.8453
  wrote registries/champion.hocon
  wrote registries/champion_manifest.hocon

That is the winner of the search -- an evolved candidate, not a seed --
rebuilt from the genome stored beside its score and served as an ordinary
neuro-san agent network. The block below predates that: until every measured
candidate carried its genome, only seeds could be resurrected, so the same
command served designer_shaped at +0.7761 while the actual champion sat
unreachable at +0.8453.

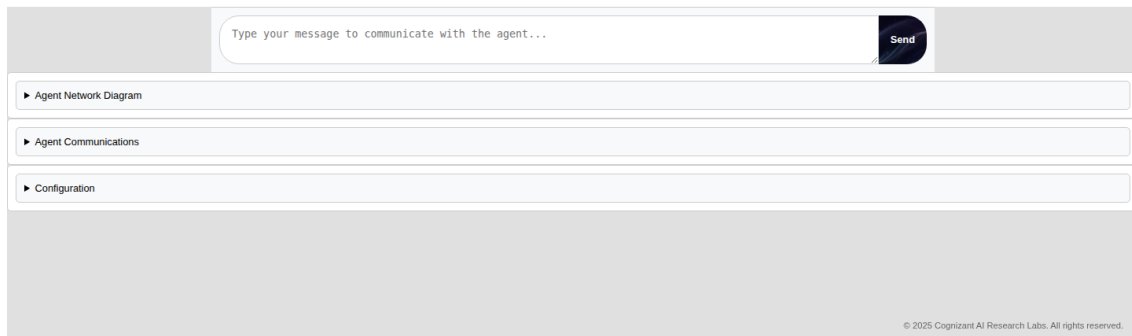
... 37 more lines
```

Captured by running it. The winner is selected from the committed measurements, written into the registry, and served by a real neuro-san server.

A real request reaches the real agents

```
origin: [{"tool": "Coordinator"}, {"tool": "IncidentSpecialist"}]
```

That is `designer_shaped` exactly as measured — the front man delegating to the specialist for an incident lookup. The topology in the report and the topology answering the question are the same object.



The neuro-san web client in Chromium, connected to the server above.



Who is the depot manager of Meridian Logistics depot D08?

Assistant is typing...

Type your message to communicate with the agent...

Send

▼ Agent Network Diagram

Not Found

The requested URL was not found on the server. If you entered the URL manually please check your spelling and try again.

▼ Agent Communications

▼ Configuration

Host: localhost

Port: 8080

Agent Network Name: champion

Update

© 2025 Cognizant AI Research Labs. All rights reserved.

The same session after sending a question. The Configuration panel shows host localhost, port 8080, agent network “champion” — the search winner, served.

It did not produce an answer, and that is not hidden

The measured model’s free-tier daily cap was spent when these were taken — 500 requests a day, resetting at midnight Pacific — so the agents returned 429 rather than a name. What is verified here is the serving path: the server loads the measured champion, a browser reaches it, and a request routes through the real agents in the real topology. The final answer needs budget that today did not have. Saying so costs a sentence; implying otherwise would cost the credibility of every other number in this document.

What running it changed

Seven things that could not have been reasoned out from the code.

The front man was the wrong agent, silently

neuro-san takes the *first* entry in a network's tool list as the agent a request enters through. Rendering agents in alphabetical order meant one seed ran with a contract specialist as its front man and another with an arithmetic agent, which answered every question with 'you did not provide any numbers'. Both scored as bad topologies. Neither had ever been run as written — every multi-agent measurement taken before this was invalid.

The cost objective had no gradient

Token cost was normalised by 60,000 while real candidates burned 260,000, so the term saturated and contributed nothing. The run would have reported itself as multi-objective while optimising accuracy alone. Fixed, the twelve measured networks spread **from 242,670 tokens to 473,450** — a factor of two — for accuracies far closer together than that. That spread is the entire opportunity, and it was invisible until the scale was right.

Rate limiting is a correctness requirement

A 429 comes back through neuro-san as an agent error. The candidate scores zero and the search learns that a perfectly good topology is bad. Pacing calls is not politeness here; it is the difference between measuring a network and measuring the provider's mood.

A blocking sleep froze every agent at once

The first limiter called `time.sleep` at an `async` call site. neuro-san runs its agents on `asyncio`, so that stopped the entire event loop: every other agent froze while still spending its own execution budget, and a queue of tasks timed out together. Caught by a lint rule, not by a test.

The first task set had no headroom

A single agent with a single tool scored 1.00 on the original 13 questions. There was nothing for evolution to improve. The corpus went from 40 documents to 124, retrieval narrowed from five results to three, and four-hop and aggregate shapes were added. An experiment whose baseline is already perfect measures nothing.

Free-tier quota is per-model and wildly uneven

Read out of the 429 payloads themselves rather than the documentation: the 'lite' models allow 500 requests per day, the full models allow 20, and the pro models allow none at all. A mutation that reassigns an agent onto a 20-per-day model is a guaranteed quota failure — which would then be scored as a bad topology. Only viable models are in the ladder.

The service was recording the wrong exhausted model

A wake that hit the daily cap reported an empty exhausted list, because it matched the error text against the failover ladder and the model that actually failed was the network default, which is not a ladder member. Every wake for the rest of that day would have spent its first calls rediscovering the same dead model. The exhausted name is now read out of the provider's own message.

Limits, and how to run it

What this does not show, said before anyone has to ask.

- **One task domain.** A topology that wins at multi-hop retrieval over a logistics corpus need not win anywhere else.
- **The Predictor is trained on tens of samples.** It ranks; it does not price. Its measured rank correlation is published in this document whatever it says, including when it says 'no better than chance'.
- **The designer baseline is a reconstruction.** It reproduces the shape `agent_network_designer` produces — one top agent, shallow DAG, fewest agents — not its literal output, because the designer wires networks against its own toolbox and cannot see this corpus. The shape is what is compared, and the shape is faithful.
- **Twelve real evaluations, a search only a couple of generations deep.** Evolved candidates did beat every seed, but on one search, one random seed, one base model and one task domain, with no repeat run and no held-out tasks. Budget arrives at three candidates a day, which is the reason the service exists and the reason this is measured in weeks rather than afternoons.
- **The surrogate has not been shown to beat picking at random.** The generation that produced the first winner ranked with a Predictor measured at -0.333 , worse than chance. The most recent network is the first the Predictor actually chose, and it is the best measured. Settling which of those the loop deserves credit for needs a run that searches with the Predictor and without it on the same budget, and that has not been bought.
- **Automated agent architecture search is an active field.** The idea is not new. What is absent from all of it is neuro-san, and what is absent from neuro-san is any fitness function at all.

Running it yourself

```
make install          # editable install with dev extras
make check            # lint + the full test suite
make offline          # phases B and C -- no key needed, no calls made

export GOOGLE_API_KEY=...
make probe            # which models have budget today
make baseline         # measure the seed topologies
make search           # the batch loop

python apps/optimizer/run_optimizer.py    # one service wake
docker compose up -d                     # the service, hourly, forever
```

Keys

No key is committed anywhere in this repository and none ever will be. Everything reads from the environment; `SECURITY.md` says so and the `.gitignore` enforces it for caches and state.

The run itself

The measurement report, carried here in full.

Evolving neuro-san agent networks

Evolutionary Surrogate-assisted Prescription applied to agent topologies — measured, not asserted.

neuro-san can turn a sentence into a working multi-agent network. It generates **one**, and never measures it: there is no fitness function anywhere in the framework. This run adds the missing half — a way to score a network, and a search for a better one.

Real evaluations	12	Surrogate evaluations	118
Best seed (baseline)	0.82 acc / 316,074 tok	Best evolved	0.94 acc / 359,600 tok

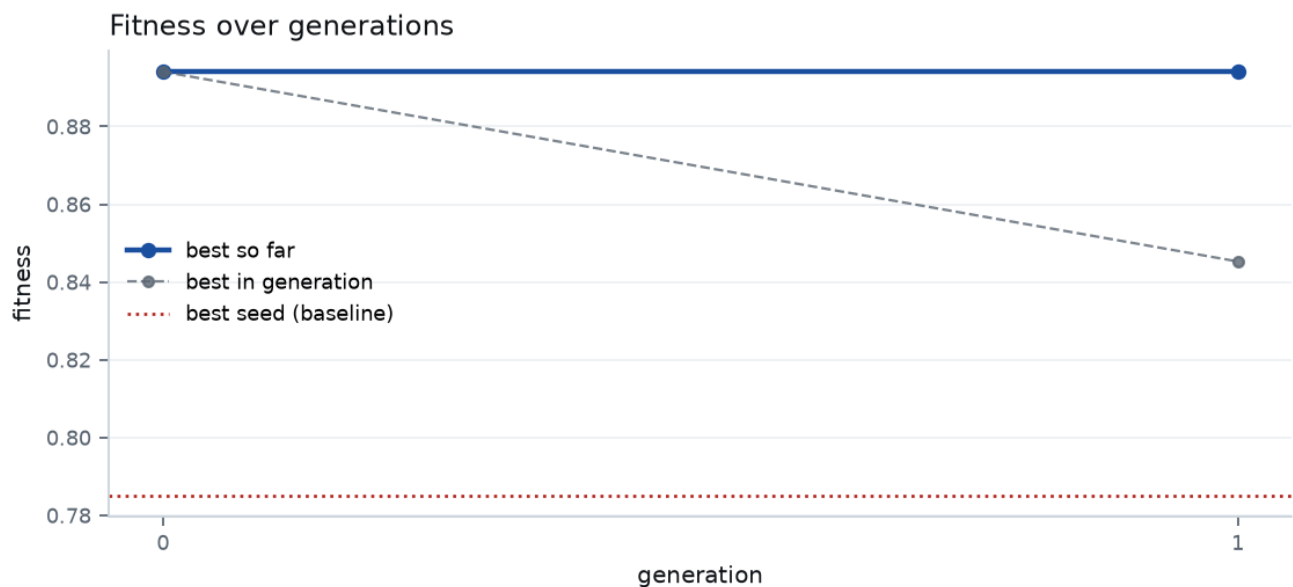
What the surrogate bought

The search examined **118** candidate networks and paid to run only **12** of them. Every candidate scored by the Predictor costs microseconds; every real evaluation costs minutes of language-model time. That ratio is the entire reason this is ESP and not a plain genetic algorithm.

The result

Evolution beat the baseline

Best evolved network: **0.94** accuracy on **359,600** tokens with 5 agents, found in generation 0 by *mut:reassign_model*. Against the best seed that is +0.12 accuracy and -43,526 tokens.



Fitness over generations. The dotted line is the best seed — the network a one-shot designer would have handed you.

Method

Four phases, repeated. Phase C is the point.

```
Phase A  seed population evaluated for real    --> (genome, fitness)
Phase B  train a Predictor on those pairs
Phase C  evolve thousands of candidates against it -- zero LLM calls
Phase D  real-evaluate only the elite, feed back to B
```

The genome is neuro-san's own `agent_network_definition` with one addition, a per-agent model. neuro-san already overlays a per-agent `llm_config` over the network default and nobody tunes it, even though it is the main cost/quality knob in a multi-agent network.

Mutation operators

Operator	What it changes
<code>add_agent / remove_agent</code>	agent count; removal re-parents children
<code>rewire</code>	one edge, at fixed size
<code>split_agent / merge_agents</code>	granularity — the axis a designer told to 'prefer the fewest agents' never explores
<code>toggle_search</code>	which agents can read the corpus
<code>reassign_model</code>	per-agent model tier

Every mutant is checked — one top agent, a DAG, no unreachable agents, no dangling references, at least one agent able to search — and **discarded if it fails, never repaired**. A repaired mutant is a different mutant than the operator produced, and silently substituting one would make the search unable to learn what its own operators do.

The evaluation world

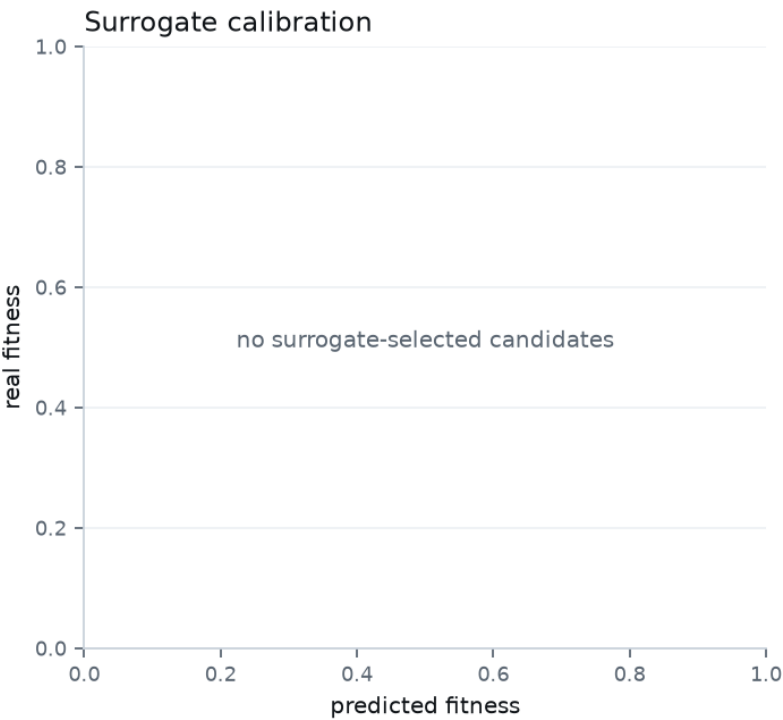
A fictional logistics company: 24 depots, 40 contracts, 60 incidents, **124 documents**, and 17 questions spanning one to four hops. The world is synthetic so that no answer can be recalled from pretraining — fitness measures the topology, not the model's memory. Corpus and ground truth come from one seeded data model, so every answer is correct by construction. Retrieval is deterministic term overlap returning three documents: fitness must be a property of the network, not of a retrieval layer that drifts between generations.

The Predictor

Whether the surrogate earned its place.

Generation	Samples	Spearman	MAE	Verdict
1	9	-0.333	0.0492	no better than chance

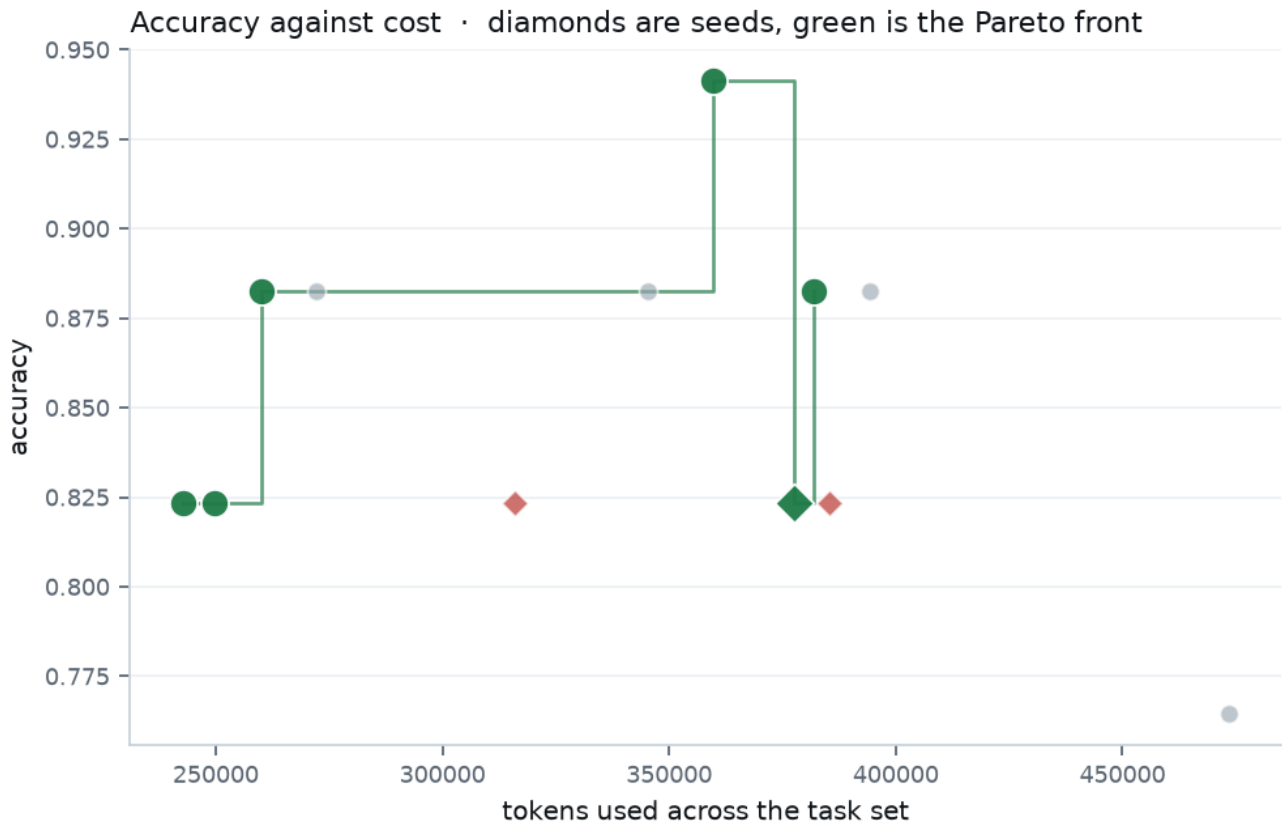
A surrogate trained on tens of samples is weak, and this one is trained on tens of samples. Its job is not to be right — it is to *rank*, well enough that the elite it selects are worth paying to run. The cross-validated rank correlation is published above whatever it says.



Predicted against real fitness for the candidates the surrogate chose. Points on the dotted line would be perfect prediction.

Accuracy against cost

A single fitness number hides the trade-off. The front does not.



Every network evaluated. Diamonds are seeds, green is the non-dominated front.

Network	Origin	Accuracy	Tokens	Agents	Depth
3bf9c008d8	mut:reassign_model	0.94	359,600	5	2
6859dda0df	mut:reassign_model	0.88	260,052	5	2
4c20dbbed3	mut:merge_agents	0.88	381,778	3	2
39fd57c6e5	mut:merge_agents	0.82	242,670	3	2
59b12db489	mut:add_agent	0.82	249,662	2	2
cbe128a617	seed:solo	0.82	377,716	1	1

Every network evaluated

The full record, in the order it was measured.

Gen	Origin	Network	Acc	Tokens	Agents	Fitness	Pred
0	mut:rewire	2a8d1782	0.76	473,450	3	+0.7107	—
0	mut:merge_agents	39fd57c6	0.82	242,670	3	+0.7926	—
0	mut:reassign_model	3bf9c008	0.94	359,600	5	+0.8941	—
0	seed:designer_shaped	459ac1a6	0.82	385,280	4	+0.7761	—
0	mut:merge_agents	4c20dbbe	0.88	381,778	3	+0.8376	—
0	mut:add_agent	59b12db4	0.82	249,662	2	+0.7941	—
1	mut:reassign_model	6859dda0	0.88	260,052	5	+0.8453	—
0	mut:split_agent	bb3340f7	0.88	394,388	5	+0.8319	—
0	seed:flat_pair	c3ee2e0c	0.82	316,074	3	+0.7852	—
0	seed:solo	cbe128a6	0.82	377,716	1	+0.7835	—
1	mut:toggle_search	cd067785	0.88	345,226	5	+0.8368	—
0	mut:split_agent	deca12e8	0.88	272,068	5	+0.8441	—

What measurement changed

Two findings that came from running it, not reasoning about it

Free-tier quota is per-model and wildly uneven. gemini-3.6-flash allows 20 requests per *day*, which cannot support an evolutionary run at all; gemini-3.5-flash-lite allows 15 per *minute*. Rate limiting became a correctness requirement, not a politeness one: a 429 returns as an agent error, the candidate scores zero, and the search learns that a good topology is bad.

The first task set had no headroom. A single agent with a single tool scored 1.00 on the original 13 questions. There was nothing for evolution to improve, so the corpus went from 40 documents to 124, retrieval narrowed from five results to three, and four-hop and aggregate shapes were added. An experiment whose baseline is already perfect measures nothing.

Limitations

- One task domain. A topology that wins at multi-hop retrieval need not win elsewhere.
- The surrogate is trained on tens of samples. It ranks; it does not price.
- The designer baseline is a faithful reconstruction of the shape agent_network_designer produces, not its literal output — the designer wires networks against its own toolbox and cannot see this corpus.
- Automated agent architecture search is an active field (ADAS, AFlow, GPTSwarm, MaAS). The idea is not new. What is absent from all of it is neuro-san, and what is absent from neuro-san is any fitness function.